Route Parameters and Query Strings in Routing

Back to: ASP.NET Core Web API Tutorials

Route Parameters and Query Strings in ASP.NET Core Web API

In this article, I will discuss **Route Parameters and Query Strings in ASP.NET Core Web API** with Examples. Please read our previous article on **Routing in ASP.NET Core Web API** Applications. When developing **RESTful services** using **ASP.NET Core Web API**, it's common for the client to send data dynamically to the server, for example:

- Fetching order details by order ID
- Retrieving a specific employee record
- Searching for products by category

ASP.NET Core provides two main ways to receive such data:

- **Route Data**
- **Query Strings**

Both mechanisms help your API **capture values directly from the request URL**, but they serve **different purposes**.

- **Route Data** is used when the value is **an essential part of the resource path**, for example, /api/employees/5 directly identifies the employee with ID 5.
- **Query Strings**, on the other hand, are used when the values are **optional filters or modifiers**, for example, /api/employees?department=HR&sortBy=name retrieves employees in the HR department sorted by name.

In short:

- Use **Route Data** for **mandatory and identity-based parameters** (like IDs).
- Use **Query Strings** for **optional or filtering parameters** (like search, filter, or sort options).

Route Data in ASP.NET Core Web API Routing

Route Data is used to pass information as part of the **URL path**. It identifies a specific resource or a subset of resources, such as an *Employee ID*, *Order Number*, or *Product Code*.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Route parameters are embedded directly in the URL and are **mandatory** for that endpoint to function. If a route parameter is missing, the route doesn't match, and ASP.NET Core returns a **404 (Not Found)** error.

In short: Use Route Parameters when the data is essential to locate or identify the resource itself.

Example to Understand Route Data in ASP.NET Core Web API:

Let's create a simple **Employee API** using *in-memory data* to simulate a database.

Define the Employee Model

Create a class file named **Employee.cs** within the **Models** folder and then copy and paste the following code:

```
namespace RoutingInASPNETCoreWebAPI.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string Department { get; set; }
        public string City { get; set; }
    }
}
```

Initialize In-Memory Data

Create a class file named **EmployeeData.cs** within the **Models** folder and then copy and paste the following code:

```
using RoutingInASPNETCoreWebAPI.Models;
namespace RoutingInASPNETCoreWebAPI.Data
{
    public static class EmployeeData
    {
        public static List<Employee> Employees = new List<Employee>
        {
            new Employee { Id = 1, Name = "Alice Johnson", Gender = "Female",
Department = "HR", City = "New York" },
            new Employee { Id = 2, Name = "Bob Smith", Gender = "Male",
Department = "IT", City = "Los Angeles" },
            new Employee { Id = 3, Name = "Charlie Davis", Gender = "Male",
Department = "Finance", City = "Chicago" },
            new Employee { Id = 4, Name = "Sara Taylor", Gender = "Female",
Department = "HR", City = "Los Angeles" },
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            new Employee { Id = 5, Name = "James Smith", Gender = "Male",
Department = "IT", City = "Chicago" },
            // Add more employees as needed
        };
    }
}
```

Fetching Employee by ID using Route Data

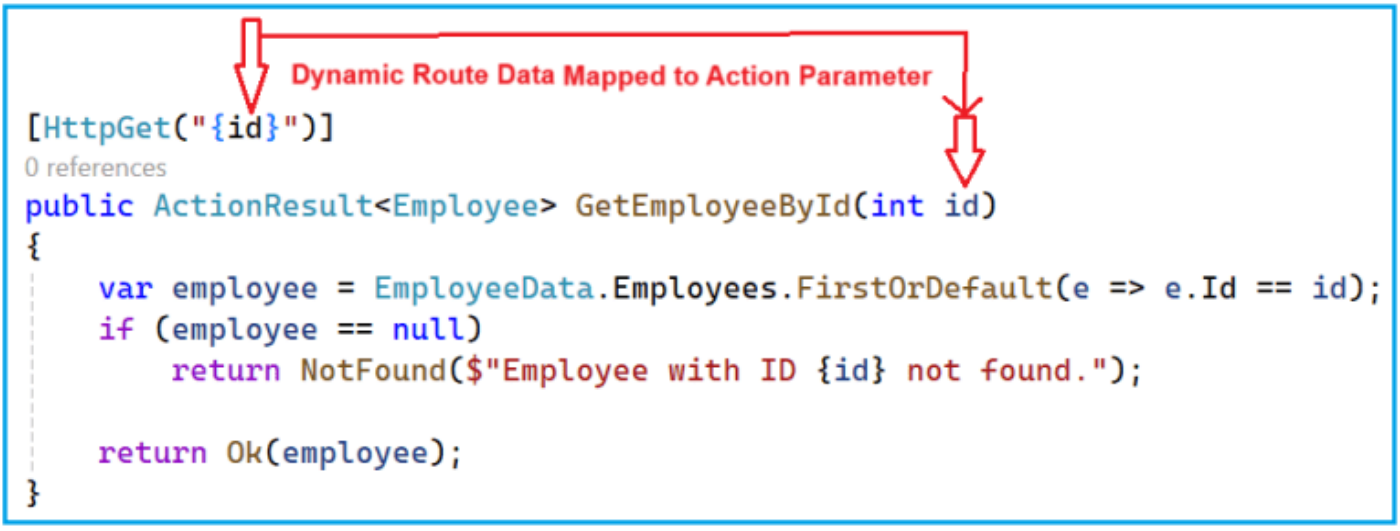
Suppose we want to fetch one employee's details by their ID. The **ID value** can naturally be part of the URL, since it uniquely identifies a resource. So, we need to define one parameter to take the ID value within the method signature, as shown in the image below.

```
public ActionResult<Employee> GetEmployeeById(int id)
{
    var employee = EmployeeData.Employees.FirstOrDefault(e => e.Id == id)
    if (employee == null)
        return NotFound($"Employee with ID {id} not found.");

    return Ok(employee);
}
```

In ASP.NET Core Web API, if you want to pass anything as part of the URL Path (Route data), you need to use curly braces **{}**. Inside the curly braces, provide the same name as the method parameter.

In our example, the GetEmployeeById method takes the id parameter, so we need to pass the id within the curly braces of the **Route or HttpGet** attribute, as shown in the image below. Here, we are using HttpGet, but you can also use the Route Attribute.



Employee Controller:

So, please modify the EmployeeController class as shown below.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/1

        [HttpGet("{id}")]
        public ActionResult<Employee> GetEmployeeById(int id)
        {
            var employee = EmployeeData.Employees.FirstOrDefault(e => e.Id ==
id);
            if (employee == null)
                return NotFound($"Employee with ID {id} not found.");

            return Ok(employee);
        }
    }
}
```

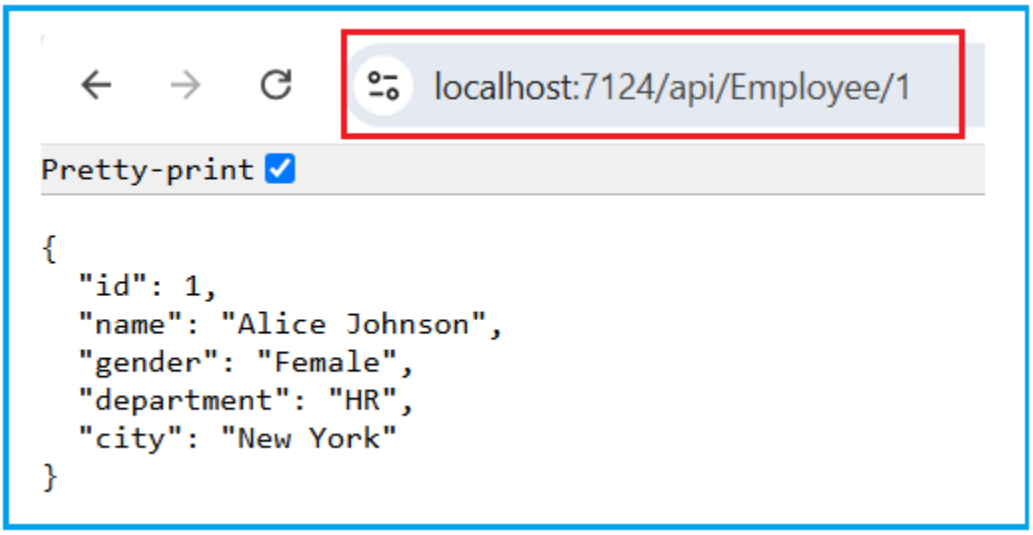
Code Explanation

- **Route Attribute:** [Route("api/[controller]")] → Base route (api/Employee) for all endpoints under EmployeeController.
- **Route Parameter:** [HttpGet("{id}")] → The {id} in the route represents a dynamic parameter.
- **Model Binding:** ASP.NET Core automatically binds the {id} value from the URL to the id parameter in the method.
- **Action Method:** GetEmployeeById retrieves the employee with the specified ID from the in-memory data.

Testing the Endpoint

Now, you can access the GetEmployeeById action method using the URL: **/api/employee/1**. If the employee exists, the API returns their details. If not, you'll receive a 404 Not Found with a custom message. So, run the above application, access the **/api/Employee/100** endpoint, and you should get the output as shown in the image below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Real-World Analogy: Think of **Route Data** like a ***house address***. If you're delivering a package, the house number (ID) is mandatory. Without it, you can't locate the right house, similar to how the API cannot find the correct resource without the route parameter.

Handling Multiple Route Parameters in ASP.NET Core Web API:

In some cases, you may need to pass **multiple route parameters**, for instance, fetching employees by both **Gender** and **City**. Here, we want the Gender to be Male or Female and the city name to be specified. So, we will create an action method that takes two parameters, both of which are string types, as shown in the image below.

```
public ActionResult<IEnumerable<Employee>> GetEmployeesByGenderAndCity(string gender, string city)
{
    var filteredEmployees = EmployeeData.Employees
        .Where(e => e.Gender.Equals(gender, StringComparison.OrdinalIgnoreCase) &&
                    e.City.Equals(city, StringComparison.OrdinalIgnoreCase))
        .ToList();

    if (!filteredEmployees.Any())
        return NotFound($"No employees found with Gender '{gender}' in City '{city}'.");

    return Ok(filteredEmployees);
}
```

Now we want to access the above GetEmployeesByGenderAndCity action method using the URL: **api/Employee/Gender/Male/City/Los Angeles**

Here, **Male** and **Los Angeles** are the dynamic values. So, we need to decorate the GetEmployeesByGenderAndCity method with the Route or HttpGet Attribute, as shown in the image below. Here, we are passing the gender and city parameters in curly braces.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
[HttpGet("Gender/{gender}/City/{city}")]
0 references
public ActionResult<IEnumerable<Employee>> GetEmployeesByGenderAndCity(string gender, string city)
{
    var filteredEmployees = EmployeeData.Employees
        .Where(e => e.Gender.Equals(gender, StringComparison.OrdinalIgnoreCase) &&
                    e.City.Equals(city, StringComparison.OrdinalIgnoreCase))
        .ToList();

    if (!filteredEmployees.Any())
        return NotFound($"No employees found with Gender '{gender}' in City '{city}'.");

    return Ok(filteredEmployees);
}
```

Modifying Employee Controller:

Please modify the EmployeeController class as shown below.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/Gender/Male/City/Los Angeles

        [HttpGet("Gender/{gender}/City/{city}")]
        public ActionResult<IEnumerable<Employee>>
GetEmployeesByGenderAndCity(string gender, string city)
        {
            var filteredEmployees = EmployeeData.Employees
                .Where(e => e.Gender.Equals(gender,
StringComparison.OrdinalIgnoreCase) &&
                            e.City.Equals(city,
StringComparison.OrdinalIgnoreCase))
                .ToList();

            if (!filteredEmployees.Any())
                return NotFound($"No employees found with Gender '{gender}' in
City '{city}'.");

            return Ok(filteredEmployees);
        }
    }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

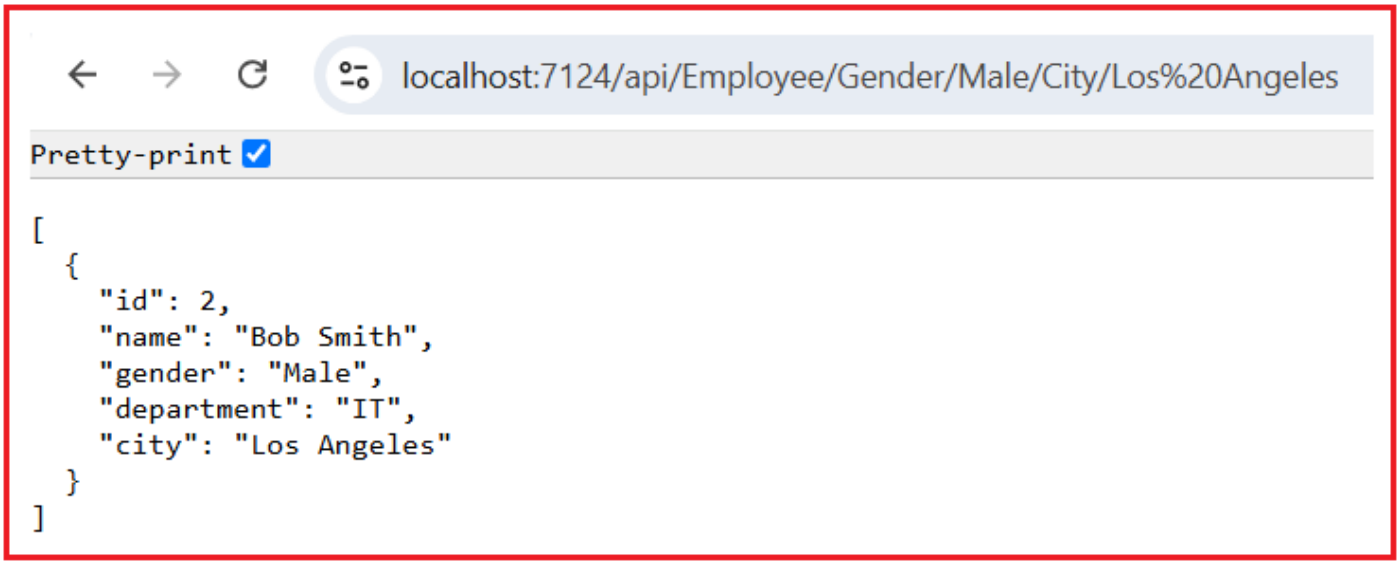
```
}
```

Code Explanation

- **Route Template:** “Gender/{gender}/City/{city}” defines two dynamic route parameters: gender and city.
- **Action Method:** Filters employees based on the provided gender and city.
- **Case-insensitive Comparison:** Ensures the search is user-friendly regardless of input casing.

Testing the Endpoint

Now run the application and navigate to the URL **/api/Employee/Gender/Male/City/Los Angeles.** You should get the message as expected, as shown in the image below.



What Is a Query String?

Query Strings are key-value pairs appended to the URL after a question mark (?). Multiple query parameters are separated by &. They typically provide optional criteria or additional information and are best suited for:

- Filtering
- Searching
- Sorting
- Paging

Query Strings don’t identify the resource itself but rather **modify the result** returned for that resource. You can specify as many optional query parameters as you want, in any order.

Searching Employees by Department using Query String in ASP.NET Core Web API

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Let us understand the query string with an example. Now, we want to search employees by **department** without making it part of the route; we can use a **query string parameter**.

Please, modify the EmployeeController class as shown below. Here, we have created a method named SearchEmployee with one parameter called Department. Further notice, we have not included that parameter in the Route Attribute.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/Search?Department=HR
        [HttpGet("Search")]
        public ActionResult<IEnumerable<Employee>> SearchEmployees([FromQuery]
string department)
        {
            var filteredEmployees = EmployeeData.Employees
                .Where(e => e.Department.Equals(department,
StringComparison.OrdinalIgnoreCase))
                .ToList();

            if (!filteredEmployees.Any())
                return NotFound($"No employees found in Department
'{department}'.");

            return Ok(filteredEmployees);
        }
    }
}
```

Code Explanation

- **Route Attribute: [HttpGet("Search")]** sets the endpoint to **api/Employee/Search.**
- **Query Parameter:** [FromQuery] tells ASP.NET Core to bind the department from the query string. Using the [FromQuery] attribute here is optional, as by default, value type parameters are bound from the Query string.
- **Action Method:** Filters employees based on the provided department. If no employee matches, it returns a 404 with a descriptive message.

Testing the Endpoint

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The URL to access the above action method is **api/Employee/Search**, but how can we pass the value for the Department parameter? The answer is the query string. So, what we need to do is append **?Department=HR** to the end of the URL (**api/Employee/Search?Department=HR**) and press the enter button as shown in the image below.

Real-Time Analogy:

Imagine visiting an **online store** (like Amazon). You open the “Laptops” category, that’s your base route. Then you add filters like **brand=Dell**, **price=under50000**, and **sort=rating**. These are **query strings** that help you filter your search results without changing the main route.

Multiple Query String Parameters in ASP.NET Core Web API

In real-world scenarios, search functionalities often require multiple optional parameters to filter data. Let’s say we want to filter employees by **City, Gender, and Department**. In that case, our action method should accept three parameters. So, modify the Employee Controller as follows. If you want to make a query string parameter optional, you need to use ? or initialize the parameter with a default value.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/Search?Gender=Male&Department=IT&City=Los Angeles
        [HttpGet("Search")]
        public ActionResult<IEnumerable<Employee>> SearchEmployees([FromQuery]
string? gender, [FromQuery] string? department, [FromQuery] string? city)
        {
            var filteredEmployees = EmployeeData.Employees.AsQueryable();

            if (!string.IsNullOrEmpty(gender))
                filteredEmployees = filteredEmployees.Where(e =>
e.Gender.Equals(gender, StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(department))
                filteredEmployees = filteredEmployees.Where(e =>
e.Department.Equals(department, StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(city))
                filteredEmployees = filteredEmployees.Where(e =>
e.City.Equals(city, StringComparison.OrdinalIgnoreCase));

            var result = filteredEmployees.ToList();

            if (!result.Any())
                return NotFound("No employees match the provided search
criteria.");

            return Ok(result);
        }
    }
}
```

Code Explanation

- **Optional Parameters:** The ? in string? makes the parameter optional.
- **Dynamic Filtering:** Filters are applied **only** when values are provided.
- **Parameter Order:** The order of query parameters in the URL does not matter.

Testing the Endpoint

With the above changes in place, now run the application.

Filter by Gender Only:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Let's say we want to filter the employees by Gender only. Then, you can only pass the Gender query string parameter in the URL **(/api/Employee/Search?Gender=Male**), as shown in the image below.

localhost:7124/api/Employee/Search?Gender=Male

Pretty-print


```
[
  {
    "id": 2,
    "name": "Bob Smith",
    "gender": "Male",
    "department": "IT",
    "city": "Los Angeles"
  },
  {
    "id": 3,
    "name": "Charlie Davis",
    "gender": "Male",
    "department": "Finance",
    "city": "Chicago"
  },
  {
    "id": 5,
    "name": "James Smith",
    "gender": "Male",
    "department": "IT",
    "city": "Chicago"
  }
]
```


Filter by Gender and Department:

Now, if you want to search employees by Gender and Department, then you need to pass two query string parameters in the URL **(/api/Employee/Search?Gender=Male&Department=IT**), as shown in the image below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

localhost:7124/api/Employee/Search?Gender=Male&Department=IT

Pretty-print


```
[
  {
    "id": 2,
    "name": "Bob Smith",
    "gender": "Male",
    "department": "IT",
    "city": "Los Angeles"
  },
  {
    "id": 5,
    "name": "James Smith",
    "gender": "Male",
    "department": "IT",
    "city": "Chicago"
  }
]
```


Filter by All Parameters:

Now, you want to search the employees by using all three parameters, i.e., Gender, Department, and City. Then, you can pass all three query string parameters in the URL **(/api/Employee/Search?Gender=Male&Department=IT&City=Los Angeles)**, as shown in the image below.

localhost:7124/api/Employee/Search?Gender=Male&Department=IT&City=Los%20Angeles

Pretty-print


```
[
  {
    "id": 2,
    "name": "Bob Smith",
    "gender": "Male",
    "department": "IT",
    "city": "Los Angeles"
  }
]
```


Using Query Strings with Complex Type in ASP.NET Core Web API

When you have many query parameters, passing them individually can make the method signature complex. Instead, you can use a **Model Class** for cleaner, maintainable code. So, let us create a model called **EmployeeSearch.cs** within the **Models** folder and then copy and paste the following code.

```
namespace RoutingInASPNETCoreWebAPI.Models
{
    public class EmployeeSearch
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    {
        public string? Gender { get; set; }
        public string? Department { get; set; }
        public string? City { get; set; }
    }
}
```

Modify the EmployeeController:

By default:

- Simple type parameters are mapped with the query string.
- Complex type parameters are mapped with the request body.

However, in our example, we want to map the complex type parameter with the Query string values. So, in this case, we need to explicitly tell the ASP.NET Core Framework to map the complex type parameter with the query string parameter by using the FromQuery attribute. So, please modify the **EmployeeController** as follows:

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/Search?Gender=Male&Department=IT&City=Los Angeles
        [HttpGet("Search")]
        public ActionResult<IEnumerable<Employee>> SearchEmployees([FromQuery]
EmployeeSearch searchCriteria)
        {
            var filteredEmployees = EmployeeData.Employees.AsQueryable();

            if (!string.IsNullOrEmpty(searchCriteria.Gender))
                filteredEmployees = filteredEmployees.Where(e =>
e.Gender.Equals(searchCriteria.Gender, StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(searchCriteria.Department))
                filteredEmployees = filteredEmployees.Where(e =>
e.Department.Equals(searchCriteria.Department,
StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(searchCriteria.City))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
                filteredEmployees = filteredEmployees.Where(e =>
e.City.Equals(searchCriteria.City, StringComparison.OrdinalIgnoreCase));

            var result = filteredEmployees.ToList();

            if (!result.Any())
                return NotFound("No employees match the provided search
criteria.");

            return Ok(result);
        }
    }
}
```

Code Explanation

- [FromQuery] EmployeeSearch searchCriteria maps all query string values to the model.
- This improves **readability** and **maintainability**.
- The approach is useful when adding future filters; simply extend the model.

Testing the Endpoint

Now, search the employees by Gender and Department as query strings in the URL (**/api/Employee/Search?Gender=Female&Department=HR**), and you should get the following result.

Combining Route Parameters and Query Strings in ASP.NET Core Web API

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

ASP.NET Core Web API allows combining both **Route Parameters** and **Query Strings** for maximum flexibility within the same endpoint. Let's consider an example where we have an API endpoint to get employee details with the following criteria:

- **Route Parameter (mandatory):** Gender (e.g., Male, Female) identifies a subset of employees.
- **Query Strings (optional):** Department and City provide additional filtering within the specified gender.

So, modify the **Employee Controller** as follows:

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // GET api/Employee/Gender/Male?Department=IT&City=Los Angeles
        [HttpGet("Gender/{gender}")]
        public ActionResult<IEnumerable<Employee>>
GetEmployeesByGender([FromRoute] string gender, [FromQuery] string? department,
[FromQuery] string? city)
        {
            var filteredEmployees = EmployeeData.Employees
                .Where(e => e.Gender.Equals(gender,
StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(department))
                filteredEmployees = filteredEmployees.Where(e =>
e.Department.Equals(department, StringComparison.OrdinalIgnoreCase));

            if (!string.IsNullOrEmpty(city))
                filteredEmployees = filteredEmployees.Where(e =>
e.City.Equals(city, StringComparison.OrdinalIgnoreCase));

            var result = filteredEmployees.ToList();

            if (!result.Any())
                return NotFound("No employees match the provided search
criteria.");

            return Ok(result);
        }
    }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
}
```

Code Explanation

- Route Parameter: {gender} is part of the route path, identifying a subset of employees.
- Query Strings: department and city provide additional optional filtering.

Testing the Endpoint

Now, with the above changes in place, run the application and search for employees by Gender as Route Data, Department, and City as query strings in the URL **(/**api/Employee/Gender/Male?Department=IT&City=Los Angeles). You should get the following result.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Understanding the difference between **Route Data** and **Query Strings** is crucial for designing clean and efficient RESTful APIs.

- Use **Route Data** for ***mandatory identifiers*** that define a specific resource.
- Use **Query Strings** for ***optional filters*** or additional criteria.
- Combine both when needed for flexibility.

Understanding and effectively utilizing Route Data and Query Strings are fundamental for building robust ASP.NET Core Web APIs. By appropriately distinguishing between mandatory and optional parameters and using the strengths of each approach, we can design APIs that are both user-friendly and maintainable.

Test

In the next article, I will discuss **Setting Multiple URLs for a Single Resource in ASP.NET Core Web API** using Routing. In this article, I explain how to work with **Route Parameters and Query Strings in ASP.NET Core Web API Routing** with Examples. I hope you enjoy the article on Route Parameters and Query Strings in ASP.NET Core Web API Routing.

---

2 thoughts on “Route Parameters and Query Strings in Routing”

SAMEERA
FEBRUARY 2, 2025 AT 11:50 PM

got clear idea about query string params and route data.
thanks sir

Reply

OUSE
FEBRUARY 25, 2025 AT 2:38 PM

genius

Reply

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Consent Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF